A ticket goes in.
A pull request
comes out.

Every claim here
has a canvas
behind it

AIForge — our own AI dev team

It writes code on machines we own. Nothing leaves the building.

Roles only — no names, no customer detail.

Where we work

Defence sector. Government customer. That is all this story needs.

Three facts about working here shape every choice in this deck:

Proof beats slides	Nothing counts until it is shown working.
Same team, more output	We grow what we deliver without growing the team.
Nothing may leave	No code or data goes to an outside service. Ever.

No product, programme or customer detail appears anywhere in this deck.



Why we built AIForge

The plan keeps growing. We want more out of the same team.

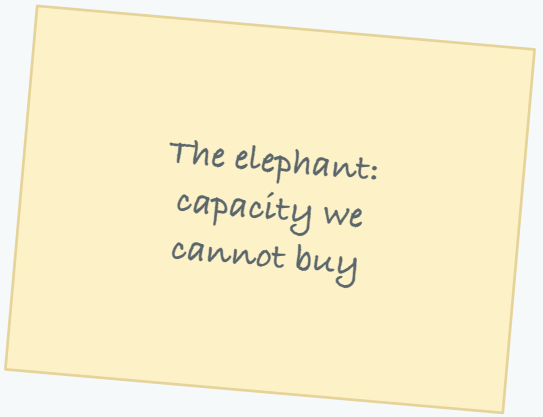
Our best engineers spend the day on small tickets. The hard work waits.

A cloud AI would help. We are not allowed to use one.

So we built our own.

It runs on our machines.

It was working before anyone approved it. That is the real story here.



Who it is for

Persona cards — roles only, no names

Delivery engineer

"I know how to do this. I do not have the hours."

Wants: the small tickets off the desk

Pain: boilerplate, stop-start days

Wins when: a pull request is waiting to be read

Junior developer

"I want to learn, not to sit blocked."

Wants: a worked example in our code

Pain: waits a day for a senior to be free

Wins when: the pull request shows the pattern

Engineering lead

"Show me it works before I plan around it."

Wants: more delivered by the same team

Pain: a claim with no number behind it

Wins when: one honest number lands each month

Persona card — write the person, not the job title

The delivery engineer is the primary persona. If their day does not change, nothing else on these slides matters.

Persona card · delivery engineer

The person the whole design is aimed at

“Half my week goes on work I already know how to do.”

Their day

- Owns a slice of a live system
- Three to six small tickets a week
- Interrupted often; deep work is rare

What they want

- Boilerplate and glue done for them
- Keep the interesting design work
- Never hand code to an outside service

What blocks them

- Reading a pull request they do not trust
- A tool that half-finishes and stays quiet
- Setup work before the tool is useful

What earns their trust

- It compiles, or it says it failed
- Says “I could not do this” in the first line
- A second model checked the work

Persona card — the objection is part of the person

Adoption test: do they file a ticket, or still walk over to a senior? Today, mostly the second.

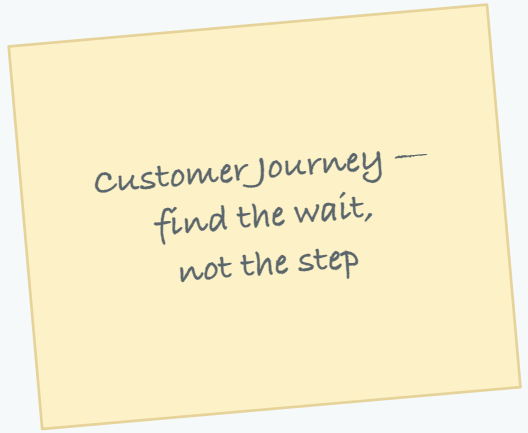
How a small change gets made

Customer journey — the same ticket, before and after

	Ticket raised	Someone picks it up	The change is written	Tests and review	Merged
Today	sits in the backlog	waits for a senior to be free	done between other work	squeezed in at the end	days after it was raised
With AIForge	picked up as written	a run starts, nobody waits	agents write it in a copy of the repo	tests first, second model reviews	a pull request is ready to read

The moment that matters: the person joins at the pull request, not before it.

The wait is the cost, not the typing. That is what this removes.



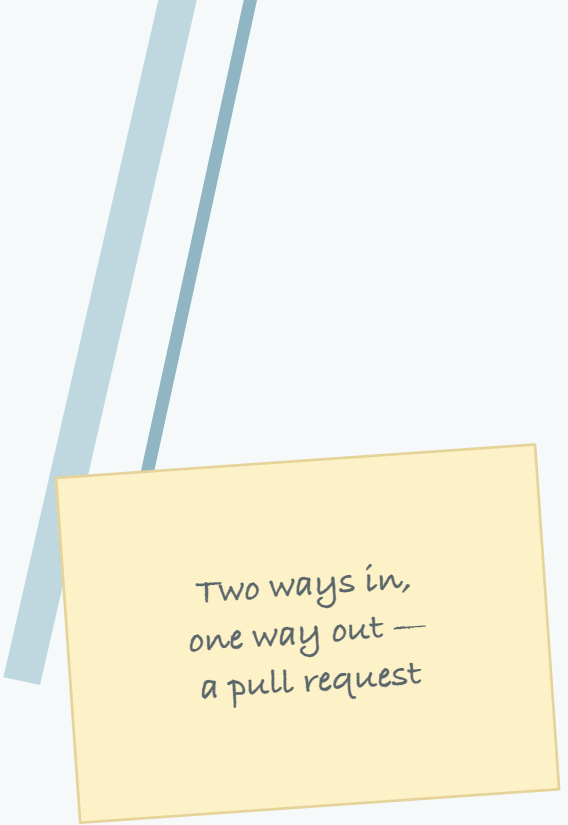
What AIForge is

Chat

You talk to it. It reads the repo, runs commands, edits files.

Ticket → PR

You file a ticket in plain words. A pull request appears. Nobody watches it work.



Four rules it follows

- A person only looks at the finished pull request.
- Only code that compiles gets merged.
- A second model checks the work.
- If it cannot do the job, it says so in the first line.

Business use case · one small change

Actor, trigger, flow, outcome — the plain version

Main flow

1. Someone files a ticket in plain words
2. The ticket is rewritten so it is unambiguous
3. The work is split; a shared spec is written
4. Each part is built in its own copy of the repo
5. Tests are written, then run against the build
6. A second model reviews the change
7. A pull request opens with an honest verdict

Actor and trigger

- Actor: delivery engineer
- Trigger: a ticket in the tracker
- Runs: several a week

It must already be true

- The ticket names a real project
- The models are loaded and warm
- The repo builds before we start

Outcome

- A pull request a person reviews
- Or a first line saying it could not
- Either way, a run history for audit

Use case card — name the actor, name the trigger

Value: the senior spends minutes reading instead of hours writing. That is the whole business case.

What it touches

Context view — the boundary around AIForge

Goes in	AIForge	Comes out
• Tickets from the tracker • The repo and its history • Past runs and lessons	• Plans the work • Writes and tests code • Reviews and merges	• A pull request • A run history for audit • New lessons stored

What we assume

- Models are loaded the right way
- Models stay warm, or runs stop mid-way
- The ticket names a real project, or routing lands wrong

What to beware of

- About 3–5 files is today's ceiling
- Some languages still beat the smaller models
- Shared GPUs limit how many tickets run at once

Context view card — write the unknowns down, not just the boxes

How one ticket flows



Each role gets a small job and only the context it needs. That is the trick.

- One shared spec file, written before any code.
- Each part builds in its own copy of the repo.
- Tests first, then the code.
- At the end it all merges and compiles until green.

Small focused context per agent — the principle behind the design

What you see, what happens behind

Service blueprint — the ticket-to-pull-request service

	Ask	Work	Check	Hand back
What the person sees	the ticket they wrote	a run in progress	test output	a pull request and a verdict
What the person does	files it, in plain words	nothing — goes elsewhere	nothing	reads, comments, merges
Front stage (visible)	ticket is picked up	branch and worktree open	compile gate runs	pull request opens
line of visibility — below this, nobody is watching				
Back stage (the agents)	enhance and clarify	plan, spec, write	test, review, reconcile	write the verdict
Behind that (platform)	ticket store and routing	model service on our GPUs	build tools and test runners	run history and lessons kept

Service Blueprint — the line of visibility is the point

Everything below the line is where the design work went. Above the line there are two moments: file it, read it.

What the business must be good at

Business capability map — shaded where AIForge already helps

Understand demand	Design the change	Build the change	Prove quality	Release and operate	Learn and improve
Capture a ticket	Read the codebase	Write the code	Write tests	Open a pull request	Keep lessons
Clarify what is wanted	Split the work	Wire files together	Run the build	Merge	Share across the team
Prioritise	Agree the contract	Keep to standards	Second-pair review	Run the platform	Measure adoption

AIForge does this today

Partly — being built

Still all people

Eleven of eighteen are done by AIForge today, three more partly. What is left is not a code gap — it is an ownership gap.

Business capability Map — what, never how

What it can do today

Capability map — and how far each one has matured

Understand the repo	Reads code, builds a map, recalls past work	Strong
Plan the work	Splits a ticket, writes the spec and contract	Strong
Write and test	Edits files, writes tests, runs the build	Strong
Check the work	Second model reviews; compile gate before merge	Good
Remember	Keeps lessons and failures across runs	Growing
Report	Honest verdict on the pull request	Good



Weakest link: memory. It works, but sharing it across the team is only now being built.

Capability card · build the change

One capability, taken apart

Turn an agreed spec into code that compiles and fits the repo it lands in.

Maturity	Good

People

- One platform operator today
- Delivery engineer reviews the output
- Nobody named for the model service

Process

- Spec and contract written before code
- Each part built in its own worktree
- Only compiling code reaches a merge

Technology

- Local models on hardware we own
- Repo map and past-run memory
- Build and test tools already in use

Gap to target

- Ceiling is 3–5 files; target is 8–10
- Memory not yet shared across the team
- Second operator not trained

Capability Card — people, process, tech — in that order

Raise it by: bigger local models after the GPU upgrade, shared memory live, a named owner for the platform.

Three things it must get right

1

Do the job on its own

Ticket in, pull request out. Nobody in the middle.

2

Make the files fit together

Change five files, they must still work as one.

3

Stay on our machines

No cloud. Works with the internet off.

Break one and the tool is not worth using. One slide each, next.



1 · Do the job on its own

HIGH

This loop is the product. Add a manual step and the reason to use it goes.

It improves the ticket, plans, writes, tests, reviews, and opens the pull request by itself.

How we keep it safe

- Tests and a second-model review are required
- It remembers past failures
- A person can steer, but never has to
- Only compiling code is merged

What it risks

- Bad automatic pull requests destroy trust fast
- A full run costs more model calls
- Licence of AI-written code needs watching



Proof: the first full run went ticket → pull request with no help.

2 · Make the files fit together

HIGH

If this fails, every big ticket fails.

Change many files and they must still work together. Imports connect, calls match, the whole change compiles as one piece.

Our local models hold about 3 to 5 files. So the work is cut up — without cutting the links.

How we do it

- One shared spec, written first
- An agreed contract between the parts
- Each part in its own copy of the repo
- Compile errors shown, never hidden

The proof

0 working lines → 53, once we added the spec and the contract.

Without it, big tickets look done and do not compile.

ASR Card:
Files fit together

3 · Stay on our machines

MEDIUM-HIGH

Shapes every model and hardware choice.

Runs on hardware we own — office GPUs, a small server, a workstation. No cloud. Works offline.

What we get

- Code never leaves the company
- No bill that grows per ticket
- Moves to another machine easily

What it costs

- Local models are weaker than hosted ones
- Load them wrong and runs fail
- Shared GPUs cap how much runs at once



Next: bigger local models after the GPU upgrade. Still ours, still inside.

What else we could have done

	Ready-made agent	One big model	Cloud AI
Nothing to build	Yes	Almost	Yes
Runs on our machines	No	Yes	No
Code stays inside	Yes	Yes	No
Handles a 5-file ticket	No	No	Yes

The first cut, before we questioned the columns

Each fails on something we cannot give up:

- The ready-made agent breaks on our model server, and cannot compile Java.
- One model in one loop cannot hold, split, test and review a multi-file ticket.
- The cloud AI works — and sends our code outside. Not allowed.

Then we saw the comparison was thin:

- It compared tools, not ways of working.
- It counted cost, not results.
- It blamed the model. The real limit is how much we hand it at once.

> ***“Just wait for a bigger model”***
>
> ***We tested it. Splitting the work beat waiting.***
>
> Same models, same machines. Only the way the work was cut up changed.



The full picture

	Ready-made	One big model	Cloud AI	Our pipeline
Code stays inside	Yes	Yes	No	Yes
Files fit together	No check	Fails	Yes	Spec + contract
Measured result	Rambles	0 linked lines	not tested	0 → 53 lines
Compiles before merge	No	No	No	Yes, every time
Who checks the work	Itself	Itself	Itself	A second model
Work to build it	None	Small	None	We built it

Now write the decision down — ADR

One column does the job on our own machines. We did not pick it because we liked it — it is the one our tests kept passing.

Decision 1 · Many small agents, not one big one

A ticket is finished by a line of specialists, each with a small job. Simple tickets skip the line.

Why

- Only option that compiles a multi-file ticket here
- Fits what small local models can hold
- Each role can be checked on its own
- A bad part fails alone

Cost

- More model calls, more waiting
- More moving parts, more ways to fail
- Capped: only real multi-file work runs the full line

ADR — options, reasons, biases, consequences

Bias we checked: “more agents must be better.” Settled with benchmark totals, not opinions.

Reversible. Owner: project lead. Decided by testing, not by meeting.

Decision 2 · Shared memory across the team

Memory sat on each machine. One person's lesson never reached anyone else.

Write	Compact	Push	Fold	Sync back
notes are plain files	tidied once a day	only what changed	server merges all authors	everyone gets the merged set

Why this way

- Files stay the source of truth — no database to run
- Duplicates are merged, so memory does not bloat as we grow
- Works alone and offline; sharing is optional
- Personal rules and workflows are never shared

Three parts still open — see the decisions log

Decisions log

What is settled, what is still open

Decision	Choice	Status
How a ticket gets built	A line of small agents, not one big one	Settled
Where models run	Our own machines only	Settled
Who checks the code	A second model, never the writer	Settled
Team memory	Local files, folded on a server, synced back	Settled
When to push memory	Once a day, after tidying	Open
When two people clash	Latest change wins, and log it	Open
How notes are grouped	Exact match first, fuzzy later	Open

ADRs keep the reasons, so nobody re-argues them next year

Everything settled here is reversible, written down, and owned by a named role.

What good looks like

Objective: make AIForge the normal way small work gets done

Key result	Target	Where we are
Tickets that end in a merged pull request	3 in 4	Not measured yet
Files one ticket can change cleanly	8 to 10	3 to 5
Tickets filed by people other than the leads	Half of all tickets	Almost none
People who can run and fix the platform	2	1
Runs where data left our network	Zero	Zero

OKR card — pick measures you can actually check

The first row is the problem. Without that number there is no case to make this standard practice — and no honest answer when the board asks.

How we prove it works

What must be true	How we check it	Where it stands
A multi-file ticket comes out as one working change	Does the merged code compile? Do the parts link?	0 → 53 linked lines; first full run green
Nothing leaves our network	A check fails the build if it would call out	Running today, small batch
We know the size limit	Files per ticket handled cleanly	3–5 files now
Team memory does not bloat	Duplicate notes; growth per week	Fold step designed

QATT card — turn a requirement into a number

Said out loud, not found per ticket: hard compile-and-fix work still needs a human finish, and shared GPUs limit how many tickets run at once.

The next four quarters

	Q1	Q2	Q3	Q4
Product	Honest verdict on every PR	Team memory sharing live	Bigger tickets, more files	Open to every team
Platform	GPU upgrade sized	Bigger local models tested	Model service run by IT ops	Capacity planned ahead
People	Second operator named	QA owns the test gate	Juniors onboarded	Adoption measured monthly

Product Roadmap Canvas — product, platform, people in one view

One rule for the whole year: nothing on this roadmap moves work off our own hardware.

What is left is people, not code

The tool works. Three jobs still have nobody's name on them.

IT / infrastructure	Watching	→	**Owning**	Run the model service — capacity, upgrades, on call.
QA & testers	Affected	→	**Owning**	Own the test gate on AI pull requests.
Platform operator	One person	→	**Two people**	Today it all stops when one person is away.

Power–Interest Grid — where people stand, and where they need to be

The people it was built for still ask a senior instead of filing a ticket. Until that changes, the case is not proven.

Next steps and what I learned

Next

- Publish one number a month: work shipped against problems caused
- Hand the model service to IT ops, with someone on call
- Train a second operator — the cheapest fix on the list
- Close the three open memory decisions
- Test bigger local models after the GPU upgrade

What I learned

- **The limit was the design, not the model.**
- The canvases made the argument checkable — one page instead of a long thread.
- A tool one person can run is not yet a tool the company owns.

Think about your own journey